Fundamentos de Aprendizagem Automática

Class 04 — Probabilistic Models

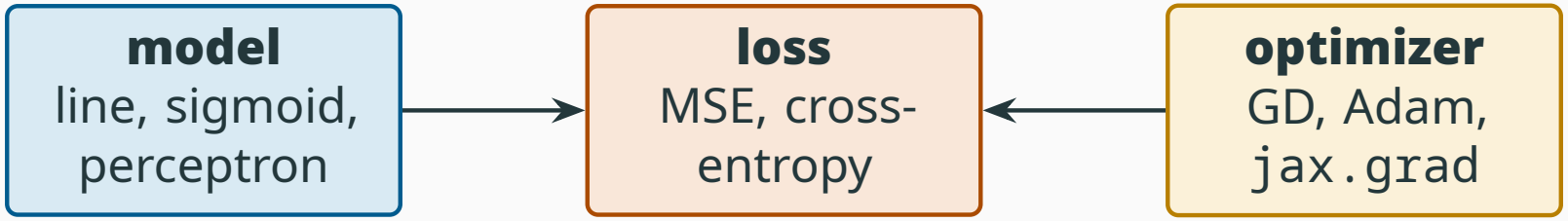
Mário Antunes

Universidade de Aveiro

2026

From Losses to Probabilities

Where We Left Off



Class 04: where do these losses come from?

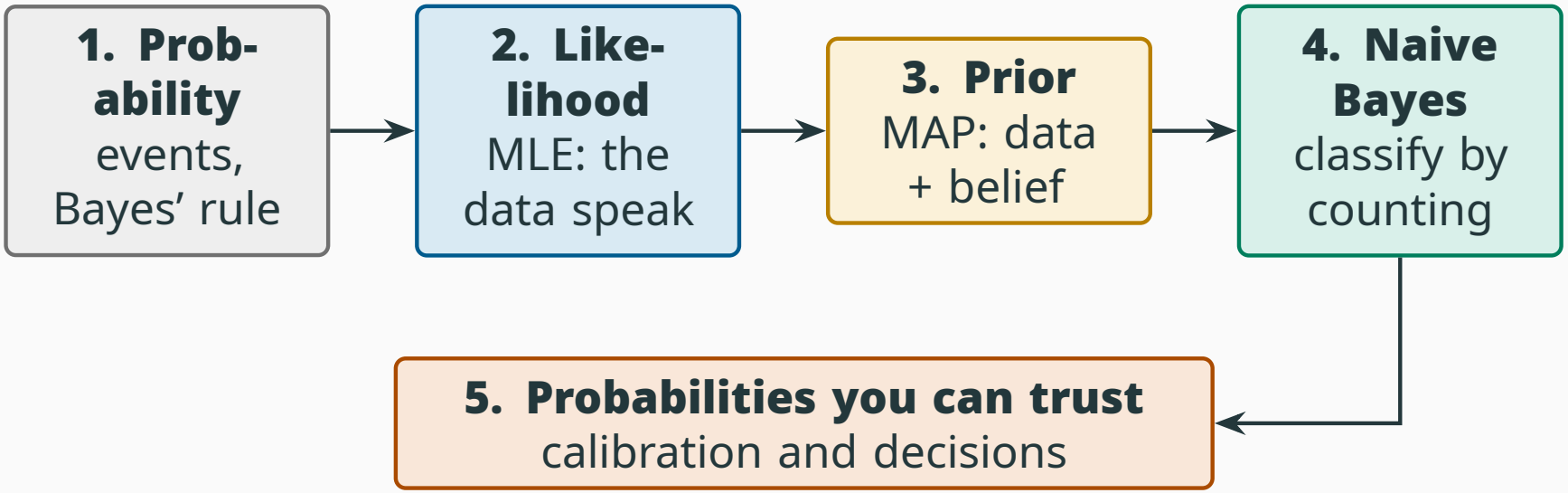
- Class 03: every model is a **model + a loss + an optimizer**; ridge and lasso add a **penalty** to the loss
- We *chose* each loss, but we never explained **why that one**
- Today's answer comes from **probability**: a loss is the *cost of a probabilistic assumption*
- On the way: a new classifier, **Naive Bayes**, that is trained by *counting*, with no optimizer

Why These Losses?

Class 03	Loss	The question
linear regression	squared error (MSE)	why *squares*, and not $\lvert e \rvert$ or e^4?
logistic regression	cross-entropy	why a *logarithm* of the probability?
ridge	MSE $+ \lambda \lVert w \rVert^2$	why penalize the *squared* weights?
lasso	MSE $+ \lambda \lVert w \rVert_1$	why $\lvert w \rvert$, and why does it give zeros?

- Class 03 gave the recipes and hinted at the answer (slides “Why Squared Error?”, “Cross-Entropy”, “The Prior View”)
- Today we build the answer from the start, assuming **no probability background**

Roadmap of the Class



- **1 to 3** answer "where do the losses come from?": a loss is a *negative log-likelihood*, a penalty is a *prior*
- **4** uses the same two ideas to build a classifier that needs no optimizer, for classes **and** for numbers
- **5** asks what accuracy hides: is "90% spam" really right 90% of the time, and what should we *do* with it?

Probability in a Nutshell

Notation: Sums, Products and Logarithms

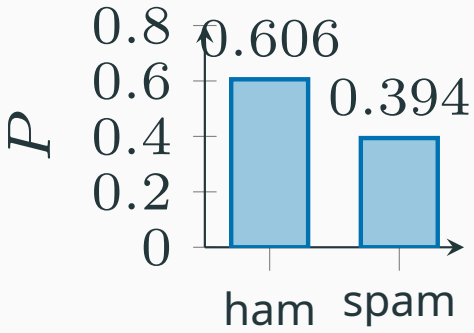
Three tools used on almost every slide of this class:

Symbol	Meaning	Example
$\sum_{i=1}^{N} a_i$	add $a_1 + a_2 + \cdots + a_N$	$\sum_{i=1}^{3} a_i = 0.5+0.2+0.1 = 0.8$
$\prod_{i=1}^{N} a_i$	multiply $a_1 \cdot a_2 \cdots a_N$	$\prod_{i=1}^{3} a_i = 0.5 \cdot 0.2 \cdot 0.1 = 0.01$
$\log a$	natural logarithm ($\log e = 1$)	$\log 0.01 = -4.61$

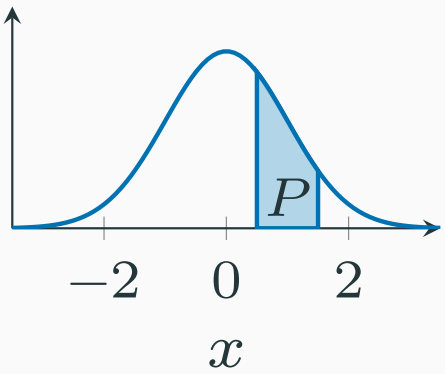
- **Logarithm turns products into sums:** $\log(a\,b) = \log a + \log b$, so $\log \prod_i a_i = \sum_i \log a_i$
- It is **increasing**: the θ that maximizes $f(\theta)$ also maximizes $\log f(\theta)$
- For $0 < p \leq 1$: $\log p \leq 0$, and $\log p \to -\infty$ as $p \to 0$

Probability: How Likely Is an Outcome?

discrete: a mass per value



continuous: an area per interval



- A **random variable** is a quantity that depends on chance: the label of an e-mail, a word count, a price. $P(A)$ is a number in $[0, 1]$ and the values add to 1
- Estimated by **frequency**: $P(\text{spam}) = 1813/4601 = 0.394$
- **Discrete** (few values): a probability *per value*. **Continuous** (a real number): a **density**; a probability is the *area* under it, and a single point has probability 0

A Distribution and Its Parameters

A **distribution** says how likely each value is; its **parameters** are the few numbers that fix its shape.

Distribution	Describes	Parameters	Probability of x
Bernoulli	yes/no (spam? word present?)	$\theta = P(x{=}1)$	$\theta^x(1-\theta)^{1-x}$
Categorical	one of M values (a bin)	$\theta_1, \ldots, \theta_M$, sum 1	θ_m
Gaussian	a number around a centre	mean μ, variance σ^2	$\frac{1}{\sqrt{2\pi\sigma^2}}\, e^{-(x-\mu)^2/2\sigma^2}$
Laplace	same, heavier tails	centre μ, scale b	$\frac{1}{2b}\, e^{-\lvert x-\mu\rvert/b}$

- **Learning a distribution = estimating its parameters from data**: this whole class
- The coin: Bernoulli with $\theta = P(\text{heads})$. The sum of the outcomes' probabilities is always 1

Joint, Marginal and Conditional

4601 e-mails, counted by class and by the presence of the word “free”:

	ham	spam	total
“free” absent	2536	824	3360
“free” present	252	989	1241
total	2788	1813	4601

- **Joint** $P(\text{free}, \text{spam}) = 989/4601 = 0.215$: both at once
- **Marginal** $P(\text{free}) = 1241/4601 = 0.270$: ignore the class (add the row)
- **Conditional** $P(\text{spam} \mid \text{free}) = 989/1241 = \mathbf{0.797}$: look **only** at the e-mails with “free”, ask the spam fraction. Without “free”: $824/3360 = 0.245$
- **Product rule:** $P(x, y) = P(x \mid y)\, P(y) = P(y \mid x)\, P(x)$

Independence

Two events are **independent** if knowing one says nothing about the other: $P(a, b) = P(a)\,P(b)$.

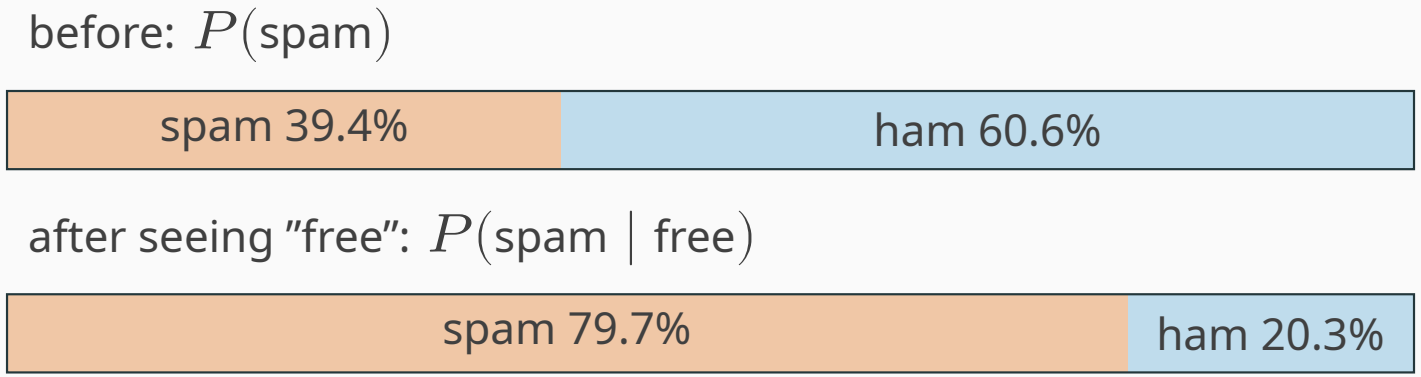
Spam data	$P(\text{free}, \text{money})$	$P(\text{free})\,P(\text{money})$
all e-mails	0.099	0.043
spam only	0.241	0.205
ham only	0.0068	0.0018

- "free" and "money" are **not independent**: seeing one makes the other more likely (both announce spam)
- Inside one class the dependence is weaker (spam: 0.241 against 0.205): **conditional independence** given the class c, $P(a, b \mid c) = P(a \mid c)\,P(b \mid c)$
- An assumption that is *false but useful* is what makes Naive Bayes work (later); here the idea matters

Bayes' Rule: Updating a Belief

Write the product rule both ways, $P(x \mid y)\,P(y) = P(y \mid x)\,P(x)$, and divide:

$$P(y \mid x) = \frac{P(x \mid y)\;P(y)}{P(x)}$$



- $P(\text{spam} \mid \text{free}) = \dfrac{0.546 \cdot 0.394}{0.270} = \mathbf{0.797}$, with $P(\text{free} \mid \text{spam}) = 0.546$
- $P(y)$ **prior**, $P(x \mid y)$ **likelihood**, $P(x)$ **evidence**, $P(y \mid x)$ **posterior** (after the evidence)

Bayesian Learning: Unknown Parameters Are Beliefs

The same rule, with the **parameters** θ as the class and the **data** D as the evidence:

$$p(\theta \mid D) = \frac{p(D \mid \theta)\, p(\theta)}{p(D)} \propto \underbrace{p(D \mid \theta)}_{\text{likelihood}} \underbrace{p(\theta)}_{\text{prior}}$$

- **Likelihood** $p(D \mid \theta)$: how probable the data are under θ. **Prior** $p(\theta)$: belief before the data. **Posterior** $p(\theta \mid D)$: after
- **MLE** (maximum likelihood estimate): the θ maximizing the **likelihood**, data alone
- **MAP** (maximum a posteriori): the θ maximizing the **posterior**, data and prior
- Class 03: a loss is — log likelihood; a penalty is — log prior

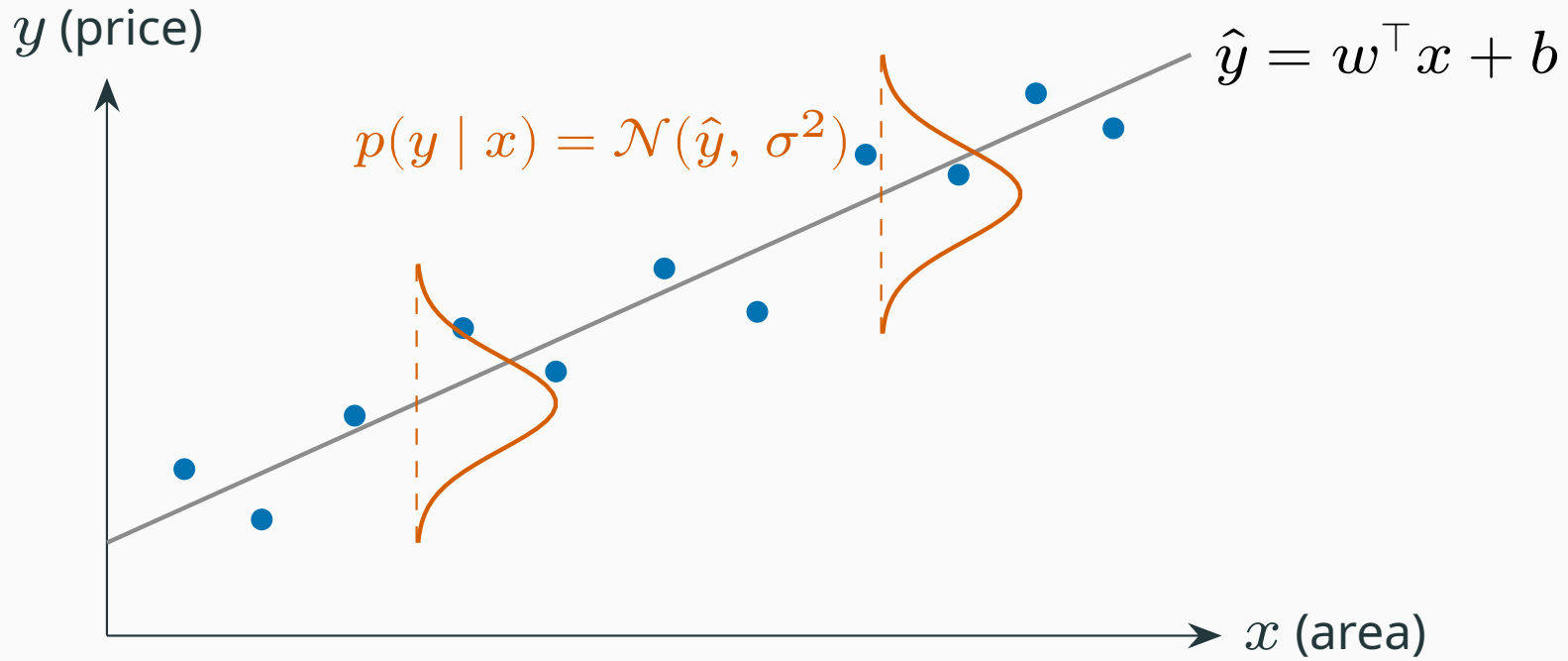
A Model Is a Distribution

A Model Gives a Distribution for y

- Class 03: a model gave **one number** $\hat{y}$ for each input x (a price, a score)
- But the real y is never *exactly* $\hat{y}$: the data hold noise, and factors we did not measure
- A **probabilistic model** says it honestly: for every input x it gives a *distribution* of the possible targets, $p(y \mid x;\, \theta)$, with parameters θ (the weights)
- The prediction is the *centre* of the distribution; its *spread* says how sure we are
- Which distribution? The one that matches the **kind of target**:

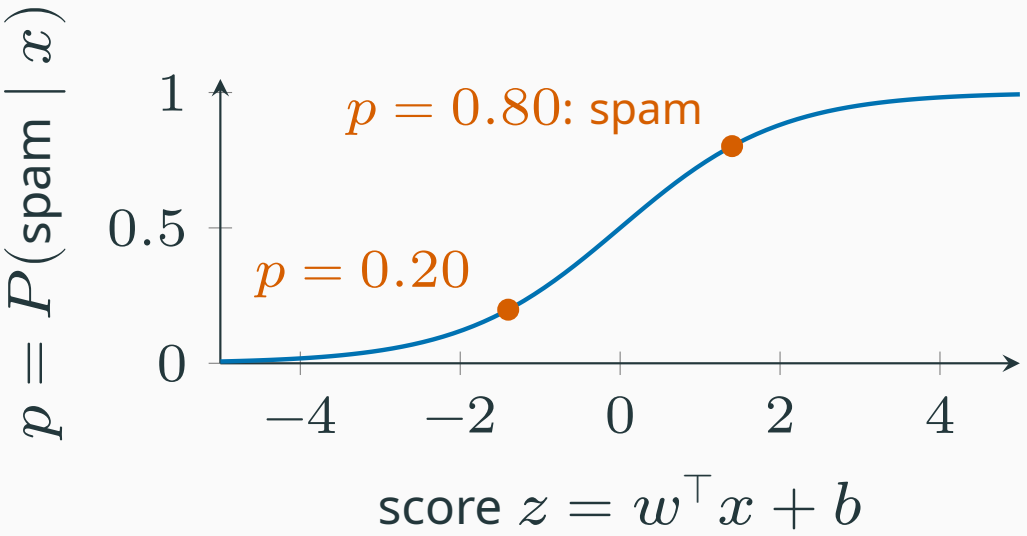
Target y	Kind	Distribution	Model
a house price	a real number	Gaussian	linear regression
spam or ham	yes/no	Bernoulli	logistic regression

Linear Regression Is a Gaussian Around the Line



- House prices of equal area still differ: the model says "around $\hat{y}$, **small** errors common, **large** errors rare"
- That is the **Gaussian** (bell): symmetric, most mass within $\pm 2\sigma$
- $y = w^\top x + b + \varepsilon,\ \varepsilon \sim \mathcal{N}(0, \sigma^2) \iff$ $y \mid x \sim \mathcal{N}(w^\top x + b,\, \sigma^2)$

Logistic Regression Is a Bernoulli with a Sigmoid



- The label is **yes/no**: the only distribution for a yes/no is the **Bernoulli**, with one parameter $p = P(y = 1 \mid x)$
- A probability must stay in $[0, 1]$, but the linear score $w^\top x + b$ does not: the **sigmoid** squeezes it (Class 03)
- $y \mid x \sim \text{Bernoulli}\big(\sigma(w^\top x + b)\big)$: the model **is** the parameter of a coin, chosen per e-mail

The Likelihood of a Dataset

The model gives a probability to the label we *actually observed*. Examples are **independent**, so the probabilities multiply:

$$L(\theta) = p(D \mid \theta) = \prod_{i=1}^{N} p(y_i \mid x_i;\, \theta)$$

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{N} \log p(y_i \mid x_i;\, \theta)$$

e-mail	label y	model says $p = P(\text{spam})$	probability of the observed label
1	spam	0.9	0.9
2	ham	0.2	$1 - 0.2 = 0.8$
3	spam	0.6	0.6

Maximum Likelihood

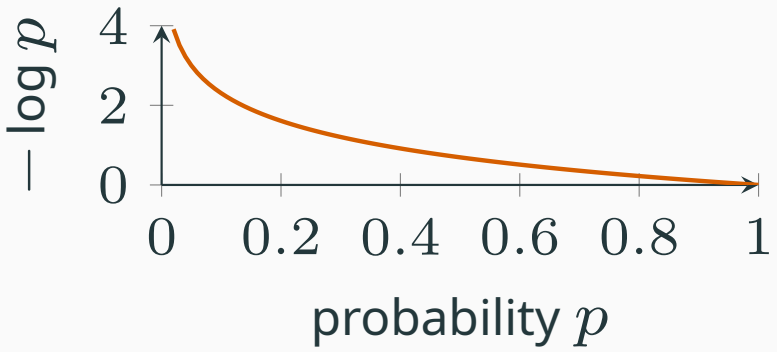
The Maximum Likelihood Estimate

The **likelihood** measures how well the parameters θ explain the data. Choose the parameters under which the data we saw are **least surprising**:

$$\hat{\theta}_{MLE} = \arg\max_{\theta} \ \ell(\theta) = \arg\min_{\theta} \ \underbrace{-\sum_{i} \log p(y_i \mid x_i;\ \theta)}_{\text{negative log-likelihood (NLL)}}$$

- Maximizing ℓ = **minimizing the NLL**, which will turn out to be **the loss** of Class 03
- Class 01: $-\log p$ is a **code length** in nats: MLE picks the model that compresses the data best
- Two ways to solve it: derivative to zero (**closed form**, often *counting*) or **descend** the NLL with `jax.grad`

Negative Log-Likelihood: the Surprise of the Data

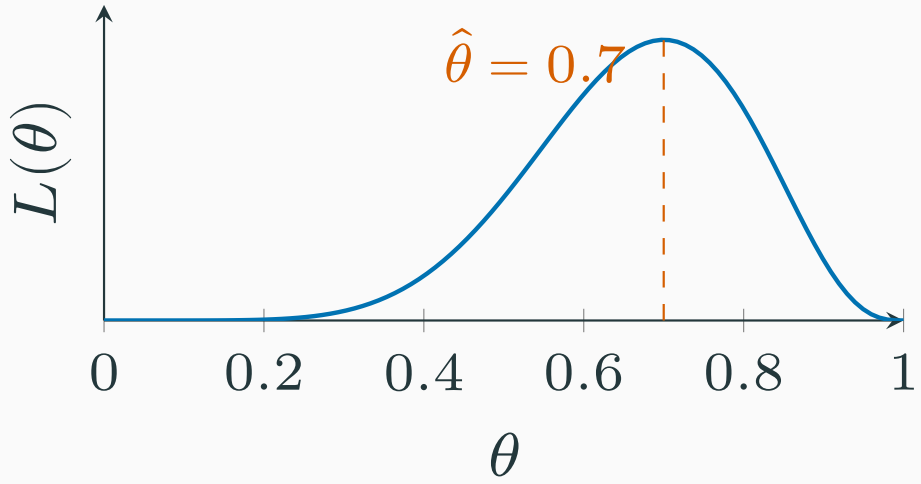


the model said	$p = 1$	0.5	0.1	0.01	10^{-6}	0
surprise $-\log p$	0	0.69	2.30	4.61	13.8	∞

- $-\log p$ is the **surprise** at an event the model gave probability p: none if certain, huge if "impossible"
- The **NLL** is the total surprise of the data: $0.105 + 0.223 + 0.511 = 0.839$ for the three e-mails
- A loss that sums a cost per example: confident and *wrong* costs a lot

Worked Example: a Coin

$n = 10$ tosses, $k = 7$ heads. Likelihood of the head probability θ: $L(\theta) = \theta^7(1-\theta)^3$



$$\ell(\theta) = k \log \theta + (n-k) \log(1-\theta), \qquad \ell'(\theta) = \frac{k}{\theta} - \frac{n-k}{1-\theta} = 0$$

- The MLE of a probability is the **relative frequency**: counting

The Same Estimate with `jax.grad`

Counting is not the only way: write the NLL and let the Class 02 machinery minimize it.

```
def neg_loglik(phi, k, n):                          # phi = logit of theta
    p = jax.nn.sigmoid(phi)
    return -(k * jnp.log(p) + (n - k) * jnp.log1p(-p))

grad = jax.jit(jax.grad(neg_loglik))
phi = 0.0
for _ in range(200):
    phi = phi - 0.1 * grad(phi, 7, 10)          # gradient descent
# jax.nn.sigmoid(phi) = 0.7000
```

- We write the negative log-likelihood; JAX gives the derivative. Optimizing the **logit** keeps $\theta \in (0, 1)$
- Same method as every model of Class 03: only the *loss* has a new name

Worked Example: a Gaussian

Five measurements $x = (2.1, 1.9, 2.6, 2.4, 3.0)$, assumed $x_i \sim \mathcal{N}(\mu, \sigma^2)$: two parameters to estimate.

$$-\ell(\mu, \sigma) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_i (x_i - \mu)^2$$

$$\implies \quad \hat{\mu} = \bar{x} = \frac{1}{n} \sum_i x_i, \qquad \hat{\sigma}^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2$$

- Derivatives set to zero: the MLE of the mean is the **average**, of the variance the **average squared deviation**
- A **closed formula**: one pass over the data, no search
- `jax.grad` on $(\mu, \log \sigma)$ agrees ($4 \cdot 10^{-16}$ apart)

A Gaussian Fit, and the Bias of the MLE Variance

	Value
$\hat{\mu} = (2.1 + 1.9 + 2.6 + 2.4 + 3.0)/5$	2.400
$\hat{\sigma}^2_{MLE}$ (divide by n): $0.74/5$	**0.148**
unbiased (divide by $n-1$): $0.74/4$	0.185

- The MLE variance is **biased**: it measures the spread around $\bar{x}$, which was fitted to *these* points, so it is too small
- On average **0.798** for a true variance of 1 with $n = 5$ (the factor $(n-1)/n$)
- A small sample misleads the MLE: a theme of this class

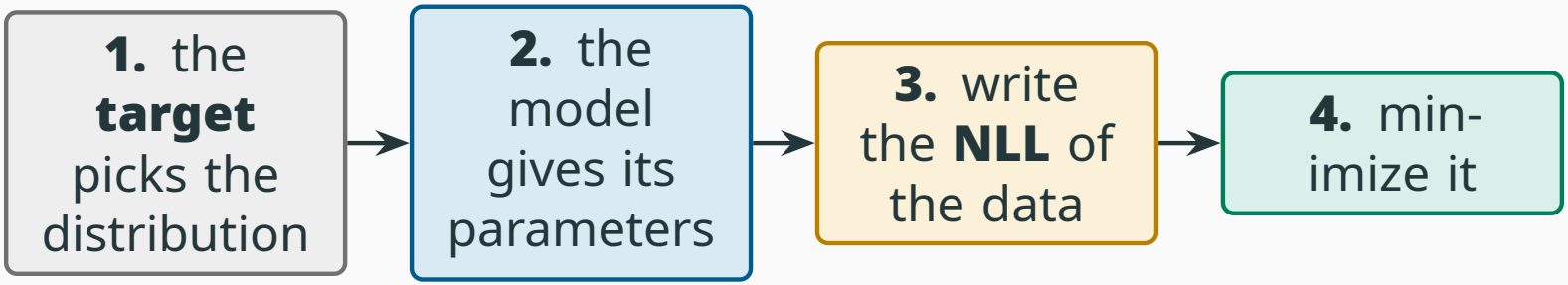
MLE: Counting, Formulas or Descending

How the minimum of the NLL is found depends on the kind of variable:

Model	Variable	MLE found by
Bernoulli, categorical, NB	**discrete**	**counting**
Gaussian mean, variance	**continuous**	a formula
linear regression	continuous	normal equation
logistic regression, nets	no closed form	**gradient descent**

- **Counting** needs **discrete** values to count: spam as "word present or not", a colour, a bin
- A **continuous** variable has none: choose a distribution (a Gaussian, ...) and estimate its parameters (Naive Bayes, below)
- No closed form $\Rightarrow$ the optimization problem of Class 02

Training a Probabilistic Model: the Recipe



- **1:** a number $\rightarrow$ Gaussian; yes/no $\rightarrow$ Bernoulli; one of M values $\rightarrow$ categorical
- **2:** mean $w^\top x + b$ (Gaussian) or probability $\sigma(w^\top x + b)$ (Bernoulli)
- **4:** counting, a formula, or `jax.grad` with a Class 02 optimizer
- The NLL **is the loss**: a different distribution gives a different loss (next two slides)

From NLL to the Squared Error

Gaussian noise: $p(y \mid x) = \dfrac{1}{\sqrt{2\pi\sigma^2}}\, e^{-(y-\hat{y})^2/2\sigma^2}$

$$-\log p(y \mid x) = \tfrac{1}{2}\log(2\pi\sigma^2) + \frac{(y-\hat{y})^2}{2\sigma^2}$$

$$\text{NLL} = \frac{N}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{depends on the weights}}$$

- Only the sum of squares depends on the weights: minimizing the NLL **is** minimizing the **MSE**
- **Why squares:** the bell makes an error of 3σ *very* rare: cost $9/2 = 4.5$ nats, against 0.5 for 1σ

From NLL to Cross-Entropy and to the Absolute Error

Bernoulli label, $p = \sigma(w^\top x + b)$:
$p(y \mid x) = p^y (1-p)^{1-y}$

$$-\log p(y \mid x) = -y \log p - (1-y) \log(1-p)$$

$\Longrightarrow$ **binary cross-entropy**; its mean over a classifier **is** `log_loss` (0.542279 = 0.542279)

Laplace noise: $p(y \mid x) = \dfrac{1}{2b}\, e^{-|y-\hat{y}|/b}$, so

$-\log p = \dfrac{|y-\hat{y}|}{b} + \log 2b \Longrightarrow$ **absolute error (MAE)**

- **Why a log:** the NLL is the surprise at the label that happened: confident and *wrong* costs a lot
- Heavier tails: an outlier is less surprising and pulls less (slope 0.523 against 0.537 for the Gaussian; true 0.50)

Every Loss Is a Negative Log-Likelihood

Noise / label model	NLL (up to constants)	Loss
Gaussian	$(y-\hat{y})^2/2\sigma^2$	**squared error**
Laplace	$\lvert y-\hat{y}\rvert/b$	**absolute error**
Bernoulli	$-y\log p-(1-y)\log(1-p)$	**cross-entropy**
Categorical (softmax)	$-\log p_y$	softmax cross-entropy

- **Choosing a loss = choosing a distribution** for the target. A new kind of target (counts, positive numbers) $\longrightarrow$ a new distribution $\longrightarrow$ a new loss (GLMs of Class 03)
- The penalties of ridge and lasso are not in this table: they come from the **prior**, in the next section

The Weakness of MLE

n	k	$\hat{\theta}_{MLE}$
3	3	**1.000**
10	7	0.700
100	70	0.700
1000	700	0.700

- Three heads in three tosses: "tails is **impossible**". A word never seen in a spam e-mail: "this word can **never** appear in spam"
- MLE fits the data it has, with no way to say "I have seen too little": **overfitting** again (Class 01)
- Remedy: encode what we believed *before* the data as a **prior**, and combine the two: **MAP**

DEMO M0–M3:

`01_mle_map.ipynb`

Maximum A Posteriori

Bayes' Rule for the Parameter

The data alone cannot say "I have seen too little". Bayes' rule adds **what we believed before**:

$$p(\theta \mid D) = \frac{p(D \mid \theta)\, p(\theta)}{p(D)} \;\propto\; \underbrace{p(D \mid \theta)}_{\text{likelihood}}\; \underbrace{p(\theta)}_{\text{prior}}$$

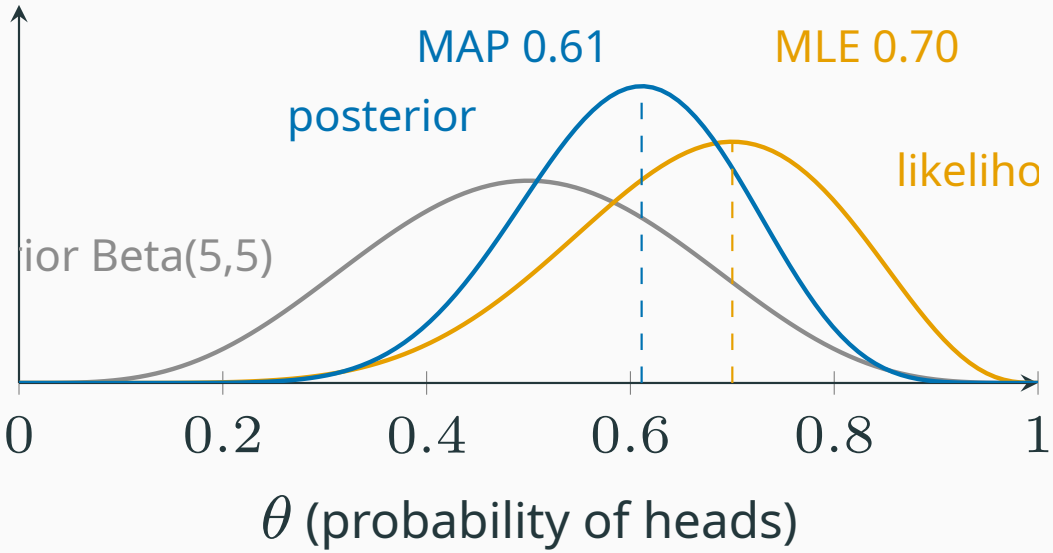
$$\hat{\theta}_{MAP} = \arg\max_{\theta} \left[\log p(D \mid \theta) + \log p(\theta) \right]$$

$$= \arg\min_{\theta} \left[\text{NLL} + \text{penalty} \right]$$

- MAP = MLE **plus a penalty** $-\log p(\theta)$: the regularization of Class 03, now with a meaning
- As N grows the likelihood (a sum of N terms) overwhelms the prior: **MAP** $\to$ **MLE**

Prior, Likelihood and Posterior: a Picture

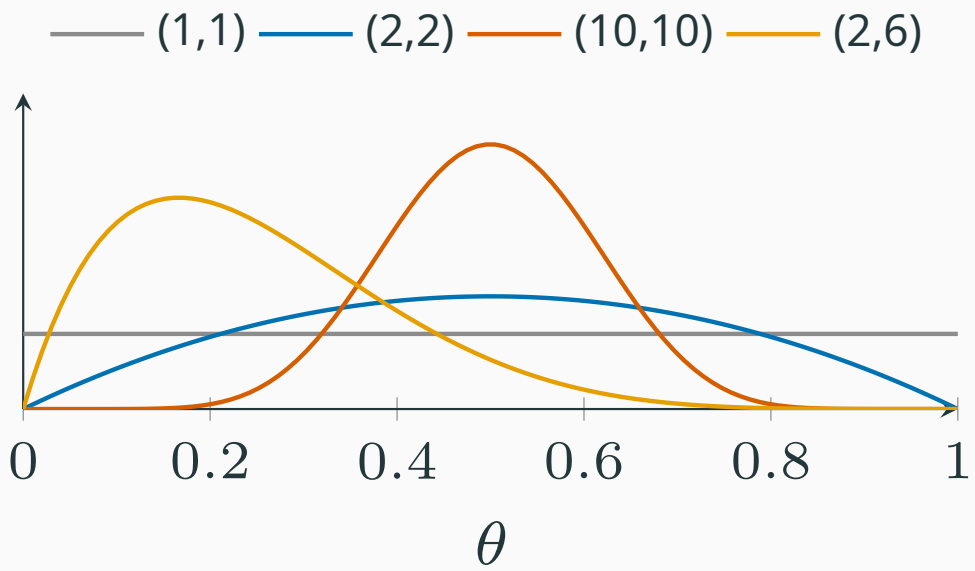
The coin again, with a prior that says "the coin is probably fair": 10 tosses, 7 heads.



- The posterior is a **compromise**: between the data (0.70) and the prior (0.50), and narrower than either
- With little data the prior pulls hard; with much data the likelihood wins (next slides)

Which Prior for a Probability? The Beta Family

A probability θ lives in $[0, 1]$, so its prior must be a density on $[0, 1]$: the **Beta(a, b)** family, $p(\theta) \propto \theta^{a-1}(1-\theta)^{b-1}$



- $a-1$ and $b-1$ are **imaginary heads and tails** seen before the data: (1,1) none, "no opinion"; (10,10) a firm belief in a fair coin; (2,6) tails likelier
- Mean $a/(a+b)$; a larger $a+b$ is a **stronger** belief

The Beta Prior Meets the Coin

Prior times likelihood:
$\theta^{a-1}(1-\theta)^{b-1} \cdot \theta^{k}(1-\theta)^{n-k} = \theta^{a+k-1}(1-\theta)^{b+n-k-1}$

$$\theta \sim \text{Beta}(a, b) \implies \theta \mid D \sim \text{Beta}(a + k,\ b + n - k)$$

$$\hat{\theta}_{MAP} = \frac{k + a - 1}{n + a + b - 2} \qquad \mathbb{E}[\theta \mid D] = \frac{k + a}{n + a + b}$$

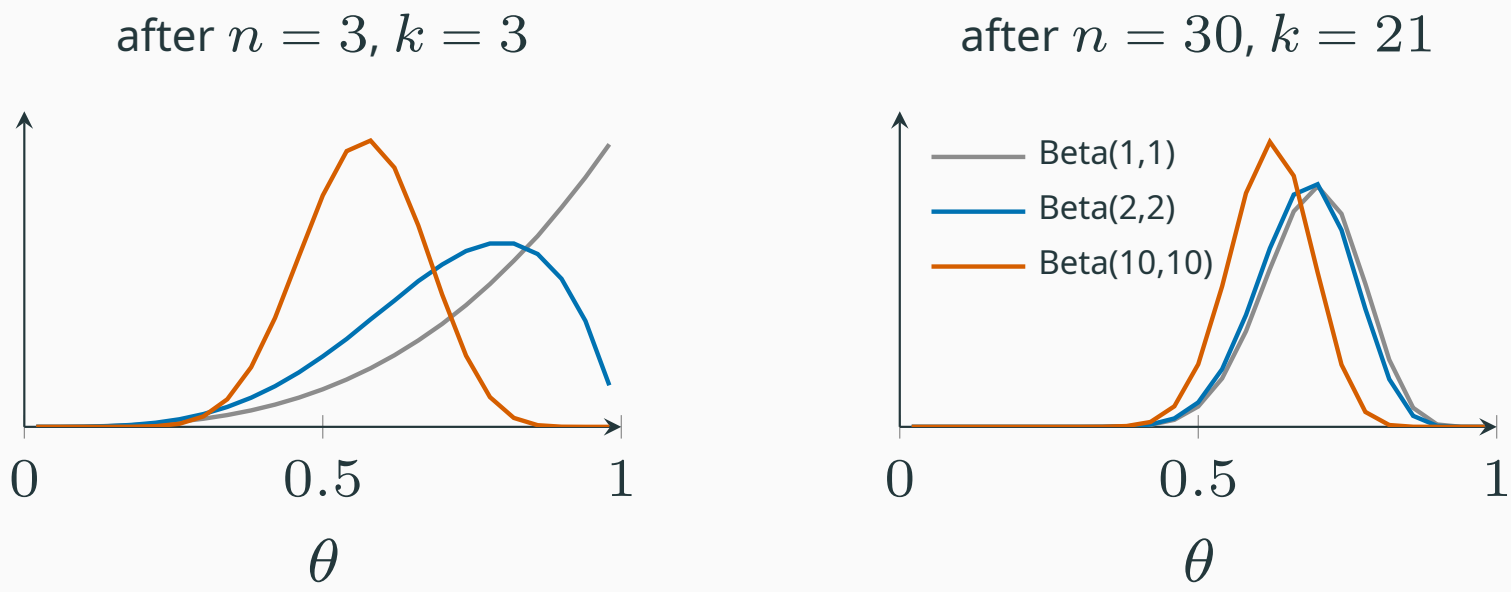
- The posterior is a Beta again: **updating = adding the counts** to a and b
- $a = b = 2$: $\hat{\theta}_{MAP} = (k+1)/(n+2)$, **Laplace's rule of succession**
- 3 heads in 3 tosses: MLE says 1.000, Beta(2,2) says **0.8**

Worked Example: the Prior Matters, then Fades

n	k	MLE	MAP Beta(2,2)	MAP Beta(10,10)	posterior mean Beta(2,2)
3	3	1.000	**0.800**	0.571	0.714
10	7	0.700	0.667	0.571	0.643
30	21	0.700	0.688	0.625	0.676
300	210	0.700	0.699	0.689	0.697

- Three heads in three tosses: the Laplace prior says **0.8** (not 1), the strong prior 0.571 (almost the prior mean 0.5)
- With 300 tosses all estimates agree: **the data win**. A strong prior needs more data to be overcome

The Posterior Shrinks Around the Data



- Posteriors under three priors: with 3 tosses they disagree (the strong prior sits near 0.58); with 30 they nearly coincide near 0.7
- The width shrinks like $1/\sqrt{n}$: more data, more certainty

Laplace Smoothing Is a Bayesian Prior

- A word never seen in a spam e-mail: the MLE $\theta = n_{cj}/n_c$ is **0**. A prior repairs the zero:

$$\theta_{cj} = \frac{n_{cj} + \alpha}{n_c + 2\alpha} = \hat{\theta}_{MAP} \text{ under a Beta}(\alpha{+}1,\ \alpha{+}1) \text{ prior}$$

A word seen in 0 of 8 spam e-mails	θ
MLE ($\alpha = 0$)	**0**
`BernoulliNB(alpha=1)` = Beta(2,2)	0.1000
`BernoulliNB(alpha=3)` = Beta(4,4)	0.2143

- α imaginary e-mails of each kind: an **assumption**, not a trick; it matches scikit-learn exactly

Ridge Is a Gaussian Prior

The ridge penalty of Class 03 from a prior: noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, weights $w_j \sim \mathcal{N}(0, \tau^2)$

$$-\log p(w \mid D) = \frac{1}{2\sigma^2} \sum_i (y_i - w^\top x_i - b)^2 + \frac{1}{2\tau^2} \|w\|^2 + \text{const}$$

$$\Longrightarrow \text{ ridge with } \alpha = \sigma^2 / \tau^2$$

- MAP by `jax.grad` equals `Ridge(alpha=sigma2/tau2)` to 10^{-15} (here $\alpha = 0.4/0.25 = 1.6$)
- The prior shrinks: weight norm 1.740 $\to$ 1.468 (100 housing districts)
- A **Laplace** prior gives the **lasso**

More Than a Point: the Posterior of the Weights

$$w \mid D \sim \mathcal{N}(\mu, \Sigma), \qquad \Sigma = \left(\frac{X^\top X}{\sigma^2} + \frac{I}{\tau^2} \right)^{-1}$$

$$\mu = \Sigma X^\top y / \sigma^2 \quad \text{(the ridge weights)}$$

Housing, 100 examples	posterior mean	posterior std
MedInc	0.621	0.127
AveOccup	-0.269	0.062
Latitude	-1.078	0.211
Longitude	-1.004	0.200

- Each weight and each prediction has an **uncertainty**: $\mathrm{var}(\hat{y}_*) = \sigma^2 + x_*^\top \Sigma \, x_*$; the 90% interval covers **0.905** of the test districts

Losses and Penalties: the Dictionary

Class 03	Loss	Probability behind it
linear regression	squared error	Gaussian noise (MLE)
robust regression	absolute error	Laplace noise (MLE)
logistic regression	cross-entropy	Bernoulli labels (MLE)
ridge	MSE + $\|w\|^2$	Gaussian **prior** on w (MAP)
lasso	MSE + $\|w\|_1$	Laplace **prior** on w (MAP)
Laplace smoothing (Naive Bayes, below)	add-one counts	Beta **prior** on a probability (MAP)

- One idea: choose the parameters that make the data **most probable** (MLE), possibly **weighted by a prior** (MAP)

DEMO M4–M5:

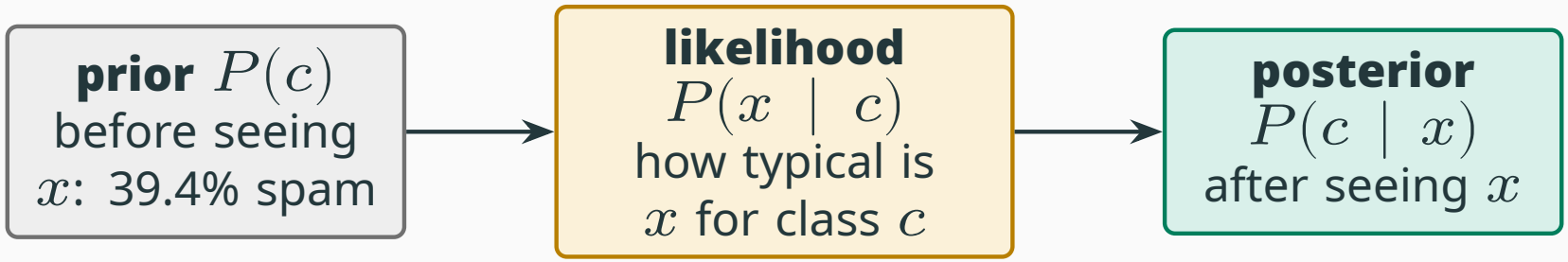
`01_mle_map.ipynb`

Naive Bayes

Bayes' Rule for Classification

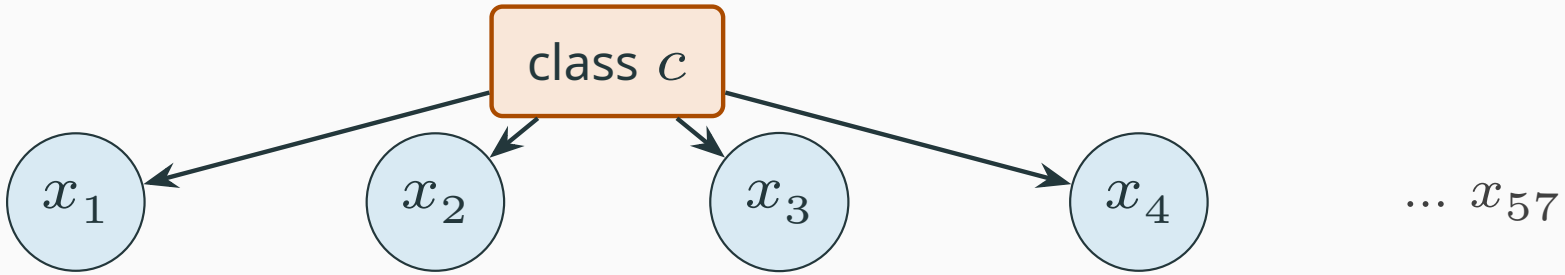
Bayes' rule with the **class** c as the unknown and the features x as the evidence:

$$P(c \mid x) = \frac{P(c)\ P(x \mid c)}{P(x)} \ \propto\ P(c)\ P(x \mid c)$$



- Choose $\hat{c} = \arg\max_c\ P(c)\,P(x \mid c)$; $P(x)$ is the same for every class and is dropped
- A **generative** model: it models how each class *produces* the features, then inverts with Bayes
- The hard part is $P(x \mid c)$ of 57 features **together**

The Naive Assumption



$$P(x_1, \dots, x_d \mid c) = \prod_{j=1}^{d} P(x_j \mid c)$$

$$\hat{c} = \arg\max_c \left[\log P(c) + \sum_{j=1}^{d} \log P(x_j \mid c) \right]$$

- Given the class, the features are **independent**: false in practice, yet it works
- A full joint over 57 binary features: $3 \cdot 10^{17}$ numbers; the naive model: **115**

Training Is Estimating Distributions

Training Naive Bayes means estimating, **per class**, a distribution for each feature:

Feature	Distribution of $P(x_j \mid c)$	Estimated by (maximum likelihood)
binary (word present)	Bernoulli(θ_{cj})	**counting**: $\theta_{cj} = n_{cj}/n_c$
categorical (a bin)	categorical	**counting** each value
continuous	a **chosen** distribution	its MLE formulas
prior	$P(c)$	**counting**: n_c/n

- n_c: examples of class c; n_{cj}: those with feature j present
- **Discrete** features are *counted*; a **continuous** one has nothing to count: *assume a shape* and fit its parameters
- **No gradient, no learning rate**: one pass over the data

Worked Example: a Six-E-Mail Corpus

e-mail	free	money	meeting	class
1	1	1	0	spam
2	1	0	0	spam
3	0	1	0	spam
4	0	0	1	ham
5	1	0	1	ham
6	0	1	1	ham

counts n_{cj}	free	money	meeting	n_c
spam	2	2	**0**	3
ham	1	1	3	3

- Priors: $P(\text{spam}) = P(\text{ham}) = 0.5$
- New e-mail: contains `free` and `meeting`, no `money`. Spam or ham?
- Note the zero: `meeting` never appeared in a spam e-mail

Worked Example: Without and With Smoothing

$\theta_{cj} = (n_{cj} + \alpha) \,/\, (n_c + 2\alpha)$ for a binary feature.
$x = (\text{free} = 1,\ \text{money} = 0,\ \text{meeting} = 1)$

	$\alpha = 0$	$\alpha = 1$
θ_{spam} (free, money, meeting)	$0.667,\ 0.667,\ \mathbf{0}$	$0.6,\ 0.6,\ 0.2$
θ_{ham}	$0.333,\ 0.333,\ 1$	$0.4,\ 0.4,\ 0.8$
spam score $P(c)P(x \mid c)$	$0.5 \cdot 0.667 \cdot 0.333 \cdot \mathbf{0} = \mathbf{0}$	$0.5 \cdot 0.6 \cdot 0.4 \cdot 0.2 = 0.024$
ham score	$0.5 \cdot 0.333 \cdot 0.667 \cdot 1 = 0.111$	$0.5 \cdot 0.4 \cdot 0.6 \cdot 0.8 = 0.096$
$P(\text{spam} \mid x)$	$\mathbf{0}$	$0.024/0.120 = \mathbf{0.2}$

- Without smoothing a single unseen word **vetoes** a class for ever: the posterior is exactly 0
- Laplace smoothing adds α imaginary occurrences of each outcome: rare is not impossible. Verdict: **ham** (posterior 0.8)

Laplace Smoothing on Real Data

An e-mail to classify may contain a word that **never appeared in one class** of the training set: with $\theta = 0$ that class gets probability **exactly 0** (an *unseen word*).

Training e-mails	$\alpha = 0$: accuracy	test e-mails with an impossible class	$\alpha = 1$: accuracy
20	0.808	807 of 921	0.836
40	0.841	663	0.860
100	0.849	551	0.885
1000	0.895	93	0.894

- With 20 training e-mails, **19 of 57** words never occur in spam: most test e-mails contain one
- Smoothing gives an unseen word a **small** probability $\alpha/(n_c + 2\alpha)$: rare is not impossible. A **MAP estimate**; it keeps the log finite. Fades with data, never for **rare** words

Floats Are Limited

A computer stores a real number in **32 or 64 bits**: a *largest*, a *smallest* and a *precision*. Smaller than the limit is rounded to **0.0**.

	`float32`	`float64`
smallest normal number	$1.2 \cdot 10^{-38}$	$2.2 \cdot 10^{-308}$
smallest of any size	$1.4 \cdot 10^{-45}$	$5 \cdot 10^{-324}$

- 57 factors of 0.05: $7 \cdot 10^{-75}$, **0.0 in `float32`**. Real spam e-mails: $P(x \mid c)$ down to $10^{-39.4}$, at the limit
- A **text** model: 200 words of probability 0.01 give 10^{-400}, **0.0 even in `float64`**
- Both classes at 0.0: the arg max is **meaningless**, with no error message

Work in Log Space

$$\log P(c \mid x) = \log P(c) + \sum_j \log P(x_j \mid c) - \log Z$$

- 10^{-400} is $\log = -921$: **sums of logs never underflow**. Log space is a *numerical-stability requirement*
- Smoothing is the other half: a probability of 0 gives $\log 0 = -\infty$, which breaks sums and gradients; it also lets **unseen words** count
- The class is the arg max of the log score. For a probability, normalize with `logsumexp`
- Bernoulli score with `jnp.where`: $\log \theta_{cj}$ if present, $\log(1 - \theta_{cj})$ if not (avoids $0 \cdot (-\infty)$)

What the Model Learned

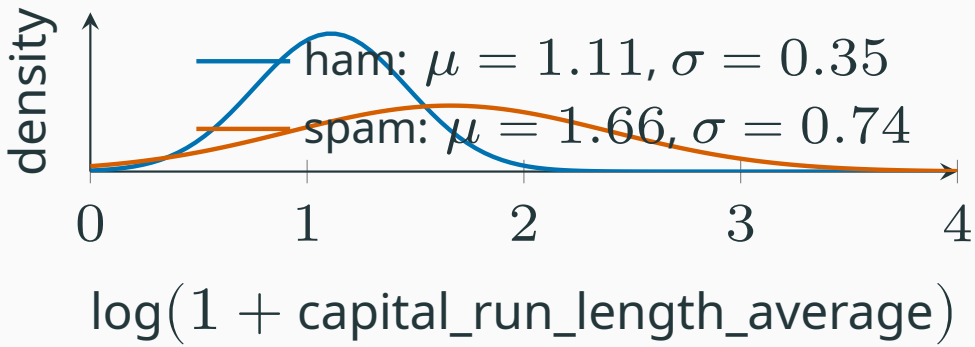
Spam words	log ratio	Ham words	log ratio
`remove`	3.27	`george`	-4.06
`money`	2.96	`cs`	-3.70
`credit`	2.60	`telnet`	-3.58
`000`	2.49	`857`	-3.52
`addresses`	2.18	`labs`	-2.80
`3d`	2.01	`hpl`	-2.75

- Log ratio $= \log P(w \mid \text{spam}) / P(w \mid \text{ham})$: each word adds evidence independently; easy to inspect
- “george”, “hpl”: the ham is the donors’ own work e-mail (UCI: `george` and area code 650 flag non-spam), a **dataset artifact** (shortcuts, Class 01)
- Bernoulli NB: test accuracy **0.893**, MCC **0.774**

Continuous Features: Gaussian Naive Bayes

$$P(x_j \mid c) = \frac{1}{\sqrt{2\pi\sigma_{cj}^2}} \exp\left(-\frac{(x_j - \mu_{cj})^2}{2\sigma_{cj}^2}\right)$$

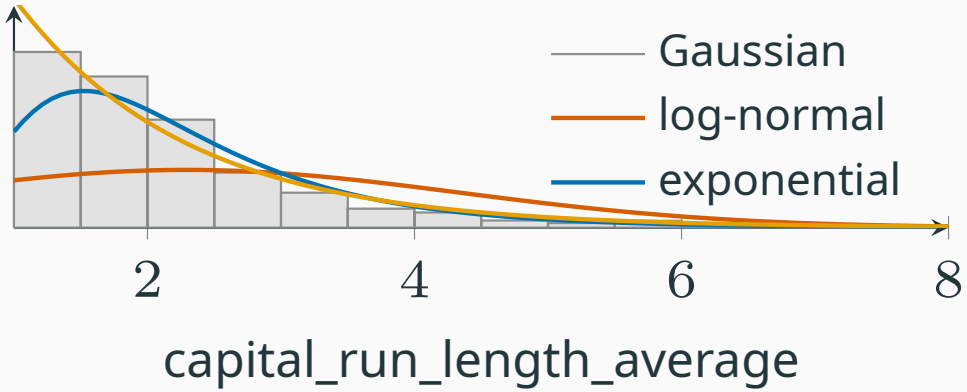
The usual choice for a continuous feature: **one bell per class**; $\hat{\mu}_{cj}$, $\hat{\sigma}_{cj}^2$ are the class mean and variance of feature j



- 2×57 means and variances; spam shouts more. Different variances: a **quadratic** boundary

Is the Gaussian the Right Bell?

Look at the histogram first. Ham e-mails, `capital_run_length_average`:



- The feature starts at 1 with a long right tail: the Gaussian wastes mass **below 1**
- Held-out log-likelihood (higher is better), ham / spam: Gaussian -15.3 / -5.0, log-normal -1.5 / -2.6, exponential -1.6 / -2.9, Laplace -2.1 / -3.4

Categorical Features: More Than Two Values

$$\theta_{cjm} = P(x_j = m \mid c) = \frac{n_{cjm} + \alpha}{n_{cj} + \alpha M} \qquad m = 1, \ldots, M$$

- A feature with M values (a bin of a numeric feature, a colour, a word count class): one probability **per value**, by counting; $M = 2$ is the Bernoulli case
- Numeric features become categorical by **binning** (quantiles): no shape is assumed
- Spam, 4 quantile bins per feature: accuracy **0.866**, MCC **0.719** (many word bins collapse to “0” and “not 0”)

Choosing the Distribution of a Feature

Spam test set (921 e-mails), 57 features	Accuracy	MCC
Gaussian NB on raw frequencies	0.845	0.716
Gaussian NB on $\log(1+x)$ (a log-normal)	0.851	0.725
exponential	0.853	0.729
Laplace	0.861	0.741
categorical, 4 quantile bins	0.866	0.719
Bernoulli, "word present"	0.893	0.774
present or not, then a log-normal for the size	**0.899**	**0.787**
logistic regression ($C = 0.1$)	*0.937*	*0.869*

- Word frequencies have **a spike at 0** and a long tail. The best NB: a **Bernoulli** for "present?" and a log-normal for "how large?"
- 921 test e-mails: differences below **0.01** are noise. The shape of $P(x_j \mid c)$ decides: plot first

Generative Versus Discriminative



- **NB** models $P(x \mid c)$: many small, easy estimates; converges fast, but plateaus at the error of its independence assumption
- **Logistic regression** models $P(c \mid x)$ directly: fewer assumptions; needs more data, then wins (Ng and Jordan, 2001)
- Close at 20 examples; from about 100 logistic is ahead (0.940 against 0.893 and 0.851). Mean of 10 random subsets

DEMO N1–N4:

`02_naive_bayes_classification.ipynb`

Naive Bayes for Regression

Naive Bayes When the Target Is a Number

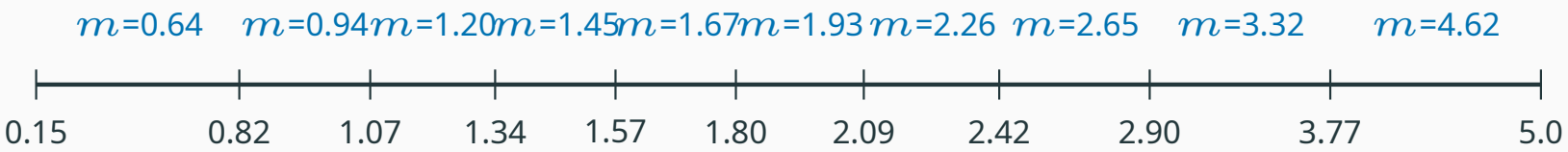
- Naive Bayes needs $P(x_j \mid y)$ and a prior $P(y)$. So far y was a **class**; nothing forces it
- House prices: y is a number. Two routes, each with **discrete** or **continuous** features:

Target	Discrete features	Continuous features
a class (spam)	Bernoulli, categorical NB	Gaussian NB
a number, **binned**	categorical NB on the bins	Gaussian NB on the bins
a number, **continuous**	bin the target (route 1)	**linear-Gaussian NB**

- **Route 1:** turn the regression into a classification of price *bins*, predict the expected price
- **Route 2:** model the features as linear-Gaussian in the price; the posterior of the price is a Gaussian, in closed form

Route 1: Discretize the Target

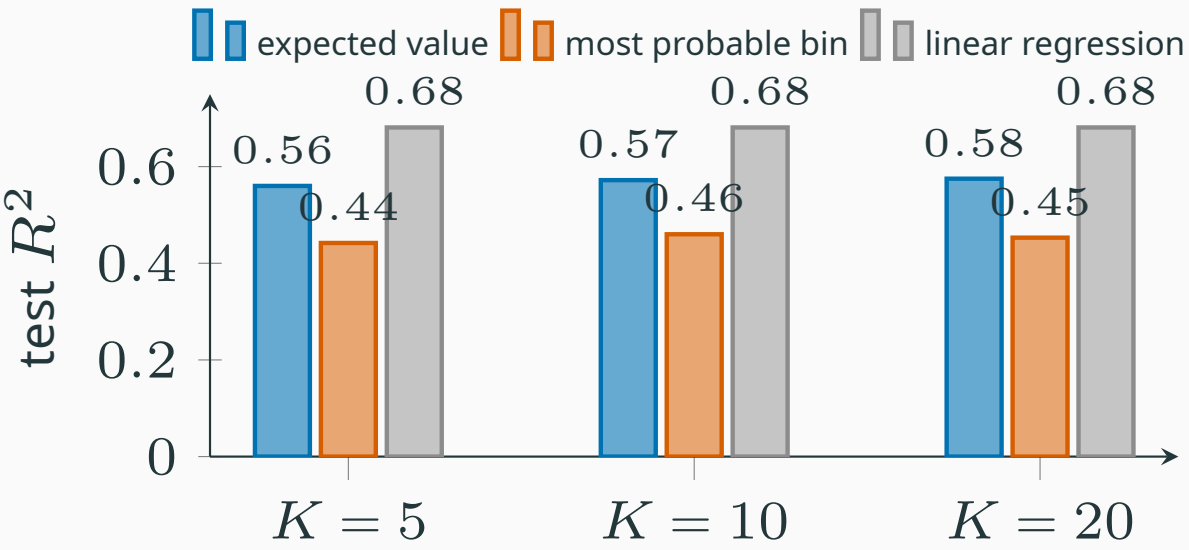
10 quantile bins of the house value (100 000 USD), about 1650 districts each; m_k = bin mean



$$\hat{y}(x) = \sum_{k=1}^{K} P(k \mid x)\, m_k \qquad \text{(expected value)}$$

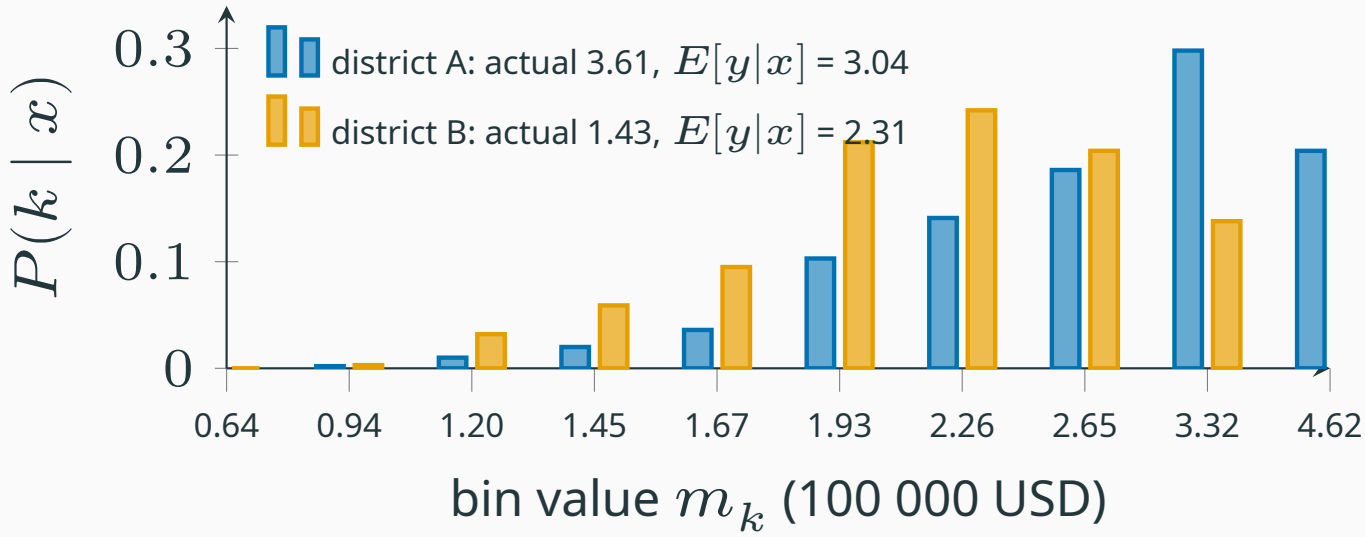
- Each bin is a **class** k with a representative value m_k (the mean of its training targets)
- Any Naive Bayes classifier gives $P(k \mid x)$; the prediction is its **expectation**
- Bins of equal size (quantiles) keep every class populated

Expected Value Versus Most Probable Bin



- Gaussian NB on the standardized housing features; test set of 4128 districts
- The **whole posterior** (the expectation) beats the most probable bin by 0.11–0.12 in R^2: the argmax can only return one of K values
- More bins help slightly; every NB version stays **below linear regression** (0.681): the features are correlated

A Distribution Over Prices for Every District



- Two test districts: the model is uncertain about both (wide posteriors), and its expectation is **off by 0.6 and 0.9**
- A bonus of the classification route: a **predictive distribution**, not just a number: probability of "more than 300 000 USD", credible intervals
- Linear regression gives a point only (unless we add a noise model)

Discrete Features for Regression

Test R^2, $K = 10$ target bins	3 bins	5 bins	10 bins	20 bins
categorical NB on binned features	0.416	0.558	0.625	**0.664**

- Each feature is cut into quantile bins; `CategoricalNB` estimates $P(\text{bin of } x_j \mid k)$ by **counting** (with Laplace smoothing)
- Too few bins lose the shape of the feature; with 20 bins the counts approximate each $P(x_j \mid k)$ **non-parametrically**: 0.664 beats Gaussian NB (0.572) and nears linear regression (0.681)
- The number of bins is a **hyper-parameter**: with more bins each cell holds fewer examples and the smoothing takes over

Route 2: a Continuous Target

$$y \sim \mathcal{N}(\mu_y, \tau^2), \qquad x_j \mid y \sim \mathcal{N}(a_j + b_j\, y,\ s_j^2)$$

(independent given y)

- MLE: μ_y, τ^2 from the targets; (a_j, b_j) by least squares of **feature on target**; s_j^2 = residual variance
- Every factor is quadratic in y: the posterior is a **Gaussian**

$$P = \frac{1}{\tau^2} + \sum_j \frac{b_j^2}{s_j^2}, \qquad \mathrm{sd}(y \mid x) = \frac{1}{\sqrt{P}}$$

$$\hat{y} = \frac{1}{P}\left(\frac{\mu_y}{\tau^2} + \sum_j \frac{b_j\,(x_j - a_j)}{s_j^2}\right)$$

Worked Example: Two Features

Prior $y \sim \mathcal{N}(2, 1)$; $x_1 \mid y \sim \mathcal{N}(1 + 0.5y,\ 0.25)$; $x_2 \mid y \sim \mathcal{N}(-1 + y,\ 1)$. Observed $x_1 = 2.4$, $x_2 = 1.2$.

	precision	vote
prior	$1/\tau^2 = 1$	$\mu_y/\tau^2 = 2$
feature 1	$0.25/0.25 = 1$	$0.5\,(2.4 - 1)/0.25 = 2.8$
feature 2	$1/1 = 1$	$1\,(1.2 + 1)/1 = 2.2$

$$P = 3, \qquad \hat{y} = \frac{2 + 2.8 + 2.2}{3} = 2.333, \qquad \text{sd} = 0.577$$

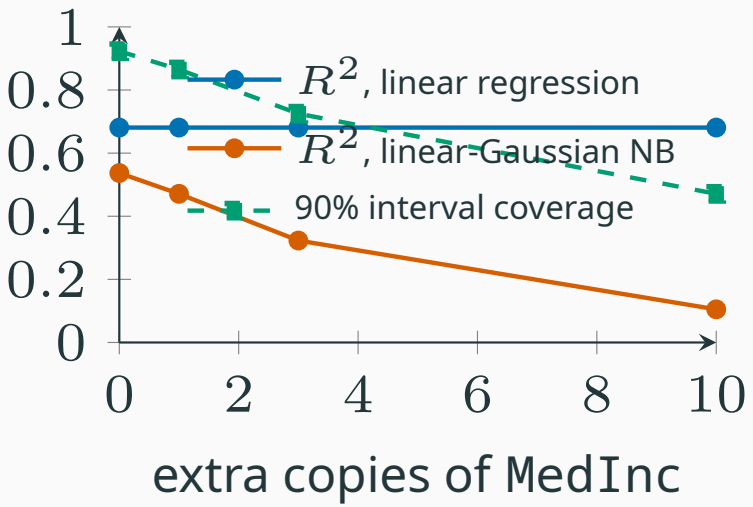
- Each source adds precision 1: the posterior is 3 times more concentrated than the prior
- The votes differ because the observations differ

Result: Linear-Gaussian Naive Bayes on Housing

Feature	b_j (per unit of y)	s_j^2	precision b_j^2/s_j^2
MedInc	0.592	0.532	**0.659**
AveOccup	-0.218	0.937	0.051
AveRooms	0.205	0.944	0.045
Latitude	-0.122	0.980	0.015
the other four			≤ 0.009

- Test $R^2 =$ **0.537**, RMSE 0.781 (linear regression 0.681, Gaussian NB on 10 bins 0.572); **median income carries most of the precision**
- Posterior sd **0.807** for *every* district (the precision does not depend on x); the 90% interval covers **0.923**, close to the nominal 0.90
- The formula agrees with a numerical maximum of the log-posterior found by `jax.grad`: 2.842745 = 2.842745 (district 0)

Where the Naive Assumption Hurts



- Append exact copies of the most informative feature: linear regression is **unchanged** (the copies share one weight)
- Naive Bayes adds the income evidence **again for every copy**: the posterior sd falls from 0.807 to **0.351** while R^2 falls to **0.105**
- With 10 copies the "90%" interval covers **47%**: over-confident *and* wrong

Naive Bayes for Regression: Summary

Model	Target	Features	Posterior	Test R^2
Gaussian NB on 10 bins	binned	continuous	distribution over bins, per district	0.572
categorical NB, 20 bins	binned	discrete	distribution over bins, per district	0.664
linear-Gaussian NB	continuous	continuous	Gaussian, same width everywhere	0.537
linear regression (Class 03)	continuous	continuous	a point	0.681

- Naive Bayes regression is cheap (counting or one least-squares per feature) and gives an **uncertainty**, but the independence assumption makes it **worse than linear regression** when features are correlated
- Use it as a fast baseline, or when a distribution over the target matters more than the last decimal of R^2

DEMO R1–R4:

`03_naive_bayes_regression.ipynb`

Probabilities You Can Trust

Naive Bayes Is Over-Confident

Model (spam test set)	Accuracy	Log-loss
logistic regression	0.940	**0.172**
Bernoulli NB	0.893	0.511
Gaussian NB	0.851	3.100

- Same data, three models: Gaussian NB is only 9 points less accurate than logistic regression but its log-loss is **18 times** larger
- It multiplies many correlated factors as if they were independent evidence: its probabilities are pushed to 0 or 1
- **Accuracy cannot see this.** A classifier must be judged as a *probability estimator* too, whenever the probability is used (costs, rankings, alarms)

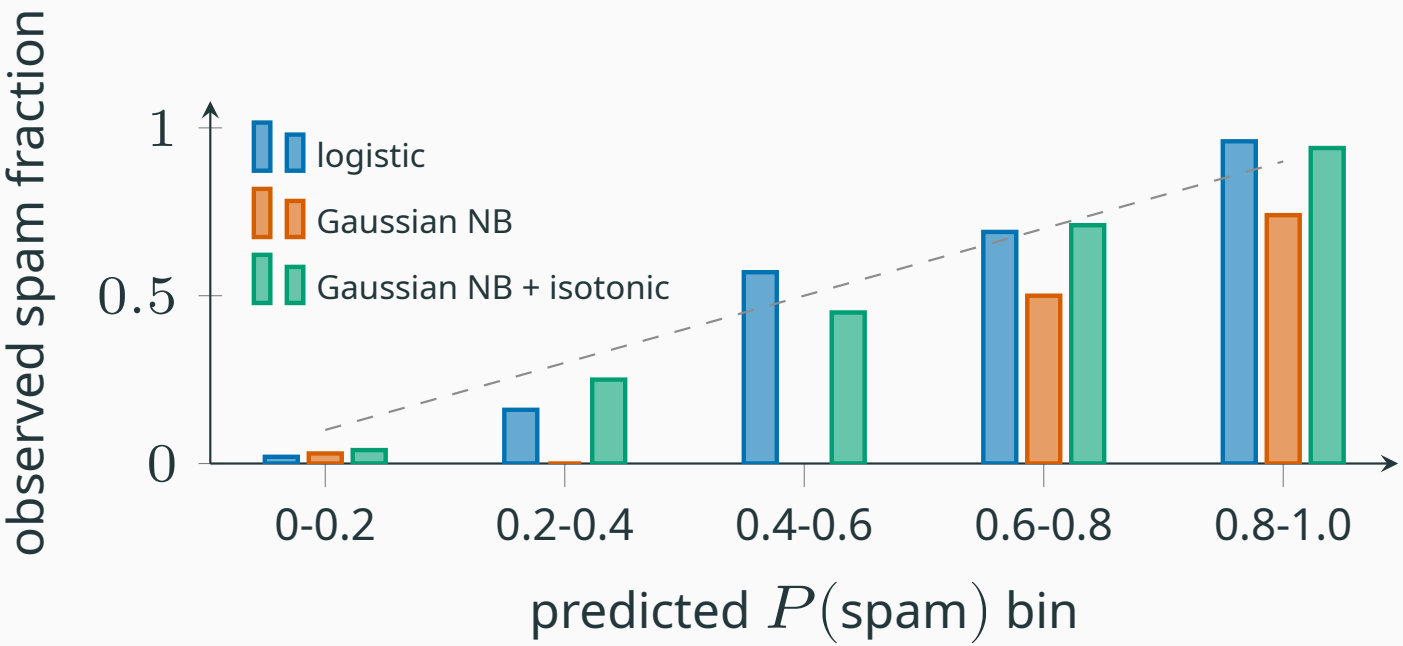
Three Ways to Measure a Probability

$$\text{log-loss} = -\frac{1}{n}\sum_i \log p(y_i \mid x_i) \qquad \text{Brier} = \frac{1}{n}\sum_i (p_i - y_i)^2$$

Spam test set	Accuracy	Log-loss	Brier	ECE
logistic regression	0.940	0.172	0.044	0.026
Bernoulli NB	0.893	0.511	0.091	0.081
Gaussian NB	0.851	3.101	0.148	0.149

- **Log-loss** = mean NLL; unbounded: one confident mistake ($p = 10^{-15}$) costs 34.5 nats
- **Brier:** the MSE of the probability, bounded in $[0, 1]$, more forgiving
- **ECE:** $\sum_b \frac{n_b}{n} \left|\text{observed}_b - \text{predicted}_b\right|$ over probability bins; lower is better

Reliability Diagram



- A calibrated model has bars near the bin centres (0.1, 0.3, 0.5, 0.7, 0.9): logistic is close, isotonic-calibrated Gaussian NB is close, **raw Gaussian NB is not**
- Raw Gaussian NB: 472 of 921 e-mails in the top bin, only **74%** spam; 445 in the lowest bin, 3% spam. Almost nothing in between

Calibrating a Classifier

- Keep the classifier; learn a **monotone map** score $\rightarrow$ probability on **held-out** data
- **Platt:** $p = \sigma(a\,s + b)$ (2 parameters). **Isotonic:** any non-decreasing step function, more flexible

Spam test set	Accuracy	Log-loss	Brier	ECE
Gaussian NB	0.851	3.101	0.148	0.149
+ Platt (sigmoid)	0.851	0.361	0.113	0.033
+ isotonic	**0.933**	**0.221**	**0.058**	0.023
Bernoulli NB	0.893	0.511	0.091	0.081
+ Platt	0.893	0.310	0.088	0.038
+ isotonic	0.891	0.285	0.077	**0.016**

- Gaussian NB log-loss 3.10 $\rightarrow$ 0.22. A monotone map keeps the **ranking**; the isotonic accuracy moves because `CalibratedClassifierCV` averages 5 fold models. Calibrate on **held-out** data

Platt Scaling with `jax.grad` (Lab F)

- The log-odds of Gaussian NB reach $-6.7 \cdot 10^8$ (median $|s| = 116$): an **over-confidence** symptom. Compress them with $t = \text{sign}(s)\log(1 + |s|)$ (monotone), then fit $\sigma(a\,t + b)$ by the cross-entropy

Lab spam test set	Log-loss	Brier	ECE	Cost (10:1) at $t^\star$
Gaussian NB, raw	3.706	0.147	0.148	1.300
Gaussian NB + Platt	**0.312**	**0.095**	0.107	**0.390**

- Fitted $a = 0.4435$, $b = -1.108$: equal to a `LogisticRegression` on the same score to $1.3 \cdot 10^{-4}$
- The log-loss falls by a factor 12 and the decision cost by 70%, without touching the classifier

Decisions Under Uncertainty

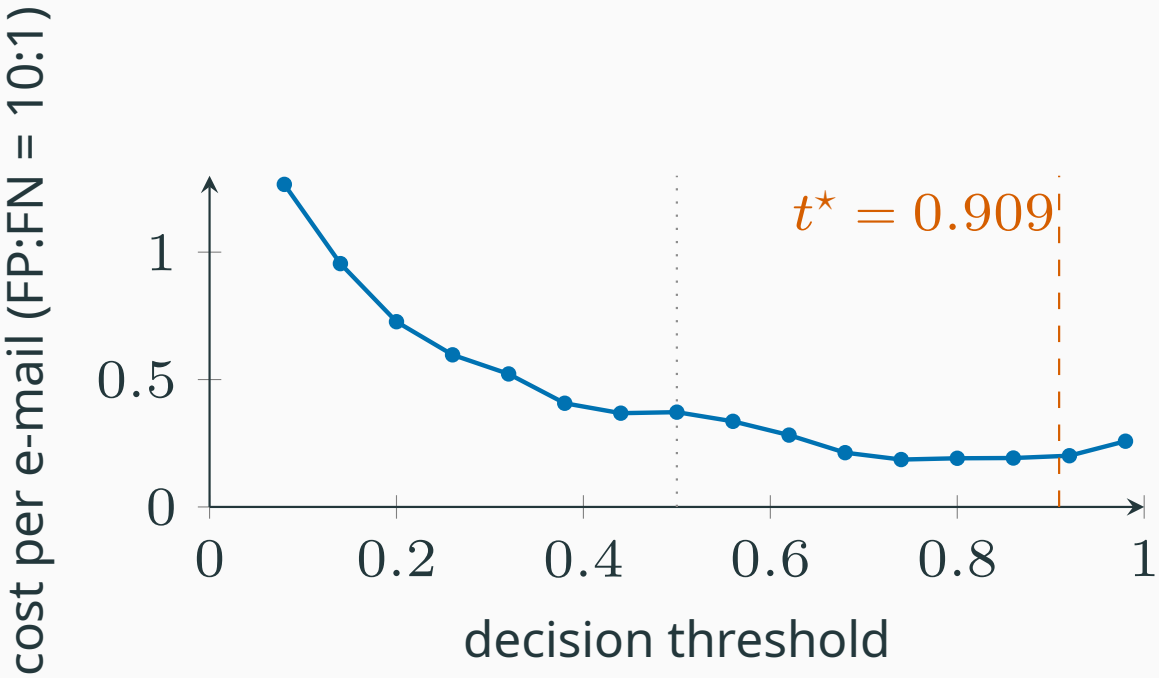
- A false positive (a real e-mail hidden) costs c_{FP}; a false negative (a spam delivered) costs c_{FN}
- Given $p = P(\text{spam} \mid x)$: flagging costs $(1-p)\,c_{FP}$ in expectation, letting it through costs $p\,c_{FN}$. **Flag when**

$$(1-p)\,c_{FP} < p\,c_{FN} \quad \Longleftrightarrow \quad p > t^\star = \frac{c_{FP}}{c_{FP} + c_{FN}}$$

Costs FP : FN	$t^\star$
1 : 1	0.500
10 : 1	**0.909**
1 : 5	0.167

- The threshold is **derived from the costs**, not tuned on the test set, **if p is calibrated**. Class 03's threshold slide gets its theory: 0.5 is right only for equal costs

Worked Example: a Real E-Mail Costs 10 Spam



Logistic, 921 test e-mails	FP	FN	Cost per e-mail
threshold 0.5	32	23	0.372
threshold $t^\star = 0.909$	8	99	**0.194**

- Hiding **24 fewer** real e-mails costs **76 more** delivered spams: worth it at 10:1. The **expected cost halves**, with the same model

Other Costs, Other Thresholds

Costs FP : FN	$t^\star$	Cost at 0.5	Cost at $t^\star$	Best scanned t (cost)
1 : 1	0.500	0.060	0.060	0.40 (0.054)
10 : 1	0.909	0.372	**0.194**	0.89 (0.180)
1 : 5	0.167	0.160	**0.140**	0.30 (0.105)

- Cost per e-mail on the test set, logistic regression. Equal costs: nothing to gain. A costly false negative (1:5) lowers the threshold: flag more
- "Best scanned" uses the test set (cheating): the gap to $t^\star$ (0.140 against 0.105 for 1:5) shows how far real probabilities are from perfectly calibrated
- Multi-class and multi-cost problems: pick the action with the **lowest expected cost** under $P(c \mid x)$ (the **Bayes decision**)

Calibrate First, Then Decide

Lab test set, costs 10:1, threshold $t^\star = 0.909$	Cost per e-mail
logistic regression (calibrated by construction)	**0.229**
Gaussian NB, raw	1.257
Gaussian NB + Platt scaling	0.390

- A threshold computed from costs is only right for **calibrated** probabilities: the raw Gaussian NB costs **five times more** than logistic regression at the same $t^\star$
- Platt scaling recovers most of the loss (1.257 $\rightarrow$ 0.390); the rest is the classifier itself (independence assumption)
- Recipe: **fit** the model $\rightarrow$ **calibrate** on held-out data $\rightarrow$ **decide** with the costs $\rightarrow$ **evaluate** the cost on the test set

DEMO N5–N6:

`02_naive_bayes_classification.ipynb`

Summary

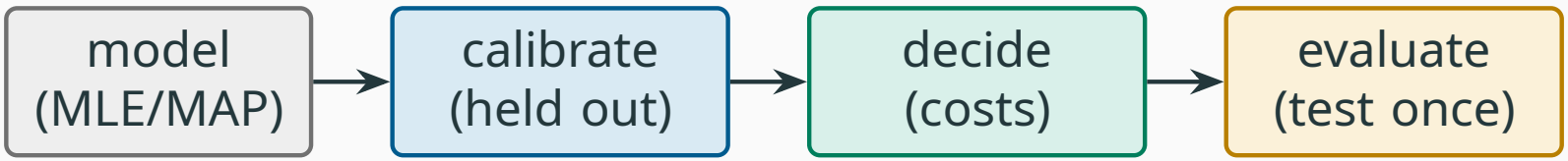
Generative Versus Discriminative, Side by Side

	Logistic	Bernoulli NB	Gaussian NB
Models	$P(c \mid x)$	$P(x \mid c)P(c)$	$P(x \mid c)P(c)$
Features	any (scaled)	binary	continuous
Training	GD on the NLL	**counting**	**means, variances**
Assumes	linear log-odds	independent	independent, Gaussian
Probabilities	good	over-confident	very over-confident
Spam accuracy	0.940	0.893	0.851

- **Little data:** few parameters, strong assumption (NB) wins; **more data:** logistic wins
- Every model is a **maximum likelihood** or **MAP** estimate

Choosing and Checking

- **A number, a class or a probability?** Regression, classification, or a probability plus a decision
- **Probabilities used for costs, ranking, alarms:** measure log-loss, Brier and reliability, not accuracy; **calibrate**, then decide
- **Fast baseline, little data, discrete features:** Naive Bayes. **Correlated features, more data:** logistic or linear regression
- **Priors:** small data or rare events $\Rightarrow$ a prior (smoothing, ridge) is not optional
- **Watch for:** zero counts, repeated or correlated evidence, thresholds left at 0.5 by habit, calibration fitted on the training data



Key Takeaways

- **MLE:** choose the parameters that make the data most probable; the losses of Class 03 are negative log-likelihoods
- **MAP:** add a prior: Gaussian $\rightarrow$ ridge, Laplace $\rightarrow$ lasso, Beta $\rightarrow$ Laplace smoothing; the prior fades with data
- **Naive Bayes:** Bayes' rule + conditional independence; **counting** for discrete features, means and variances for continuous ones
- **Regression too:** discretize the target and classify (expected value), or model the features as linear-Gaussian in y
- **Over-confidence:** independence counts repeated evidence again; log-loss, Brier and reliability expose it; Platt or isotonic scaling repairs it
- **Decisions:** flag when $p > c_{FP}/(c_{FP} + c_{FN})$, if p is calibrated

Lab 04: Build It in JAX

Part	Task	twin
A	Probability and Bayes' rule; MLE and MAP with `jax.grad` (Gaussian, Beta, ridge)	`Ridge`, `BernoulliNB`
B	Bernoulli NB, discrete features	`BernoulliNB`
C	Gaussian, exponential and categorical NB	`GaussianNB`, `CategoricalNB`
D	NB regression: binned and continuous target	`GaussianNB`, `CategoricalNB`
E	log-loss, Brier, reliability; costs	`log_loss`, `calibration_curve`
F	challenge: Platt scaling	`LogisticRegression`

- Guide `practice/04_probabilistic_models.pdf`, notebook `lab04_probabilistic_models.ipynb`, solution in `solutions/`
- **Project 1** is due at Class 08: use the methodology of Classes 03 and 04: visualize, split, preprocess on training only

References (1/2)

- C. Bishop, *Pattern Recognition and Machine Learning*, 2006: chapters 1 and 2 (probability, MLE, Bayesian estimation), 4 (classification)
- D. Barber, *Bayesian Reasoning and Machine Learning*, 2012: chapters 9 (learning as inference: MLE, MAP) and 10 (Naive Bayes)
- G. James, D. Witten, T. Hastie, R. Tibshirani, J. Taylor, *An Introduction to Statistical Learning with Applications in Python*, 2023: section 4.4 (generative models for classification, Naive Bayes), 4.5 (comparison of classifiers)
- P. Domingos, M. Pazzani, “On the optimality of the simple Bayesian classifier under zero-one loss”, *Machine Learning*, 1997

References (2/2)

- A. Ng, M. Jordan, “On discriminative vs. generative classifiers”, *NIPS*, 2001
- J. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods”, 1999
- B. Zadrozny, C. Elkan, “Transforming classifier scores into accurate multiclass probability estimates”, *KDD*, 2002
- A. Niculescu-Mizil, R. Caruana, “Predicting good probabilities with supervised learning”, *ICML*, 2005
- C. Elkan, “The foundations of cost-sensitive learning”, *IJCAI*, 2001